Lecture 4 — Sorting, Custom Comparators & Greedy Basics

> Year Curriculum · Prerequisites: Weeks 1–3 (STL, complexity, complete search) · Estimated time: 2 hours lecture + 4 hours practice

1. Overview

Sorting is the most useful preprocessing step in competitive programming. Once data is in order, equal elements sit next to each other, the smallest and largest are at the ends, binary search becomes possible (Week 5), two pointers become possible (Week 6), and many **greedy** algorithms become obvious. This lecture covers:

1. What sorting costs, and the special cases where you can do better.
2. Sorting compound data: pairs, structs, and indices. Writing **correct** custom comparators.
3. The greedy paradigm: make the locally best choice and never look back.
4. How to **prove** a greedy correct with an exchange argument, and how to find counterexamples when it is wrong.
5. A set of classic greedy problems and the sort key that solves each.

2. Sorting

2.1 Cost

Any algorithm that sorts using only comparisons needs Ω(n log n) comparisons in the worst case. The algorithm must distinguish all n! input orders, each comparison yields one bit of information, and $\log_2(n!) \approx n \log_2 n - 1.44n$. `std::sort` (introsort: quicksort that falls back to heapsort) achieves O(n log n) in the worst case. Sorting 2·10^5 integers takes a few milliseconds, and 10^6 takes well under 100 ms.

- `std::sort` is **not stable**: equal elements may be reordered.
- `std::stable_sort` (merge sort) is stable, also O(n log n).
- `std::nth_element(a.begin(), a.begin() + k, a.end())` places the k-th smallest element at position k, with smaller elements before it and larger after, in **O(n)** average time. Use it for medians.
- **Counting sort**: when values lie in a small range [0, K), count occurrences in O(n + K) and output them in order.

2.2 Sorting pairs and structs

`pair` and `tuple` compare lexicographically, so `sort` on `vector<pair<int,int>>` orders by `.first` and breaks ties by `.second`. Two common tricks:

- **Choose the key by field order.** To sort intervals by end time, store them as `{end, start}`.
- **Remember the original position** by storing `{value, index}`. After sorting you still know where each value came from, which helps when the output must be in input order.

For structs, either define `operator<` or pass a lambda.

2.3 Sorting indices (argsort)

When several parallel arrays describe the same items, or when you must not lose the original order, sort a permutation of indices instead:

```
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0);
sort(idx.begin(), idx.end(), [&](int i, int j) { return a[i] < a[j]; });
```

Now `idx[k]` is the index of the k-th smallest element. Inverting this permutation (`rank[idx[k]] = k`) gives each element's rank, which is the core of **coordinate compression**: replacing values up to 10^9 by their ranks 0..n−1 so they can index an array. Regional lecture 6 uses this heavily.

2.4 Comparators must be strict weak orderings

A comparator `cmp(x, y)` answers "must x come strictly before y?" The standard library requires:

1. **Irreflexivity**: `cmp(x, x) == false`.
2. **Asymmetry**: if `cmp(x, y)` then `!cmp(y, x)`.
3. **Transitivity**: `cmp(x, y) && cmp(y, z)` implies `cmp(x, z)`.
4. **Transitivity of equivalence**: if x ~ y and y ~ z (neither is before the other), then x ~ z.

The most common violation is using `<=`. With many equal elements, `std::sort` may then run past the end of the array, and the result is a **runtime error**, not just a wrong order. A subtler violation is a comparator on floating-point values containing NaN, or one that compares with an epsilon ("equal if within 1e-9"), which breaks transitivity of equivalence.

Multi-key comparisons are easiest with `std::tie`, which builds a tuple of references:

```
return tie(x.score, y.name) > tie(y.score, x.name);  // score desc, then name asc
```

Placing `y.name` on the left side reverses the direction for that key only.

2.5 An unusual comparator: largest concatenation

Given non-negative integers, arrange them to form the largest possible concatenated number. Neither numeric order nor plain lexicographic order works: "3" vs. "30" gives "330" > "303", so 3 must come first, yet "3" < "30" lexicographically. The right rule is that **x comes before y iff x + y > y + x** as strings. It can be shown that this relation is a strict weak ordering: treat each string s as the rational number s / (10^|s| − 1), and the comparison becomes numeric comparison of those values. Hence sorting with it is valid, and an exchange argument on adjacent elements proves the result optimal.

3. The greedy paradigm

A **greedy algorithm** builds a solution step by step, at each step making the choice that looks best right now and never revisiting it. Greedy solutions are short and fast, typically a sort followed by one scan, in O(n log n). The danger is that they are often **wrong**, and wrong greedies usually pass the samples.

3.1 A greedy that fails

Make change for 6 with coins {1, 3, 4}, using as few coins as possible. "Always take the largest coin that fits" gives 4 + 1 + 1 (three coins), but 3 + 3 uses two. The greedy is correct for the coin systems used by most real currencies, but not in general. The general problem needs dynamic programming (Week 14).

Before trusting a greedy, spend two minutes looking for a small counterexample, or stress-test it against a brute force (Week 3).

3.2 Proving a greedy: the exchange argument

The standard proof technique:

1. Let G be the greedy's solution and O an optimal solution.
2. Look at the first decision where they differ.
3. Show that O can be modified to agree with G at that decision without becoming worse, usually by **swapping** two elements or replacing one choice with the greedy's.
4. Repeating this transforms O into G without loss, so G is optimal.

A second technique is "**greedy stays ahead**": show by induction that after k steps, the greedy's partial solution is at least as good as any other solution's first k steps, by some measure.

4. Classic greedy problems

4.1 Interval scheduling — CSES 1629 Movie Festival

There are n movies with start and end times. Maximize the number of pairwise non-overlapping movies you can watch fully.

Greedy: sort by end time. Scan, taking each movie that starts at or after the end of the last movie taken.

Proof (exchange). Let g be the movie with the earliest end time and O an optimal schedule whose first movie is o. Since end(g) ≤ end(o), replacing o with g in O creates no conflict: every other movie in O starts after end(o) ≥ end(g). The new schedule is still optimal and starts with g. Remove g and every movie overlapping it, and apply the same argument to the remaining problem.

Wrong keys to avoid. Sorting by start time fails on a long movie that starts first and blocks many short ones. Sorting by length fails when a short movie overlaps two disjoint long ones.

Worked example. Movies (3,5), (4,9), (5,8):

- Sorted by end: (3,5), (5,8), (4,9).
- Take (3,5); lastEnd = 5.
- (5,8) starts at 5 ≥ 5, so take it; lastEnd = 8.
- (4,9) starts at 4 < 8, so skip it. Answer: 2.

Complexity: O(n log n) for sorting plus O(n) for the scan.

4.2 Pairing — CSES 1090 Ferris Wheel

Children have weights p_i. Each gondola holds at most two children with total weight ≤ x. Minimize the number of gondolas.

Greedy: sort. Consider the heaviest remaining child. If they fit with the lightest remaining child, put the two together. Otherwise the heaviest rides alone, since they fit with nobody. Repeat.

Proof sketch. If the heaviest child h can share with anyone, h can share with the lightest child l. Take an optimal solution where h is not with l. Suppose h rides with a, or alone, and l rides with b, or alone. Rearrange to {h, l} and {a, b}, dropping a or b if absent. {h, l} fits by assumption. {a, b} fits because b ≤ h, so a + b ≤ a + h ≤ x. The number of gondolas does not increase. Two pointers implement this in O(n) after sorting.

4.3 Minimizing the sum of distances — CSES 1074 Stick Lengths

Choose L to minimize Σ|p_i − L|. **The median minimizes it.** If L has k points to its left and n − k to its right, moving L right by δ (without crossing a point) changes the cost by δ·(k − (n − k)). The cost decreases while fewer than half the points are to the left, and increases afterwards. With even n, any L between the two middle elements is optimal. Use `long long` for the total, since it can reach 2·10^5 · 10^9.

4.4 Minimizing total completion time — CSES 1630 Tasks and Deadlines

Tasks have duration a_i and deadline d_i. Do them all in some order. A task finishing at time f earns d − f, which may be negative. Maximize the total.

The total is Σd_i − Σf_i. The first sum does not depend on the order, so we **minimize the sum of finish times**. **Shortest job first** is optimal. **Exchange argument:** take two adjacent tasks, a long one (length L) followed by a short one (length S < L). Swapping them leaves every other task's finish time unchanged. Before the swap, their finish times sum to (t + L) + (t + L + S). After, they sum to (t + S) + (t + S + L). The sum decreases by L − S > 0. Hence, in an optimal order, durations are non-decreasing. The deadlines turn out to be irrelevant to the order, a classic trap.

4.5 Feasibility with deadlines — ABC131 D Megalomania

Jobs with duration A_i must each finish by B_i, one at a time, starting at time 0. Is it possible?

Earliest deadline first (EDF): sort by deadline, run the jobs in that order, and check every finish time. **Exchange:** if some feasible order runs job j (later deadline) immediately before job i (earlier deadline), swap them. Job i now finishes earlier, which is fine. Job j finishes when i used to finish, which is ≤ B_i ≤ B_j, also fine. So if any order is feasible, EDF is.

4.6 Sweeping events — CSES 1619 Restaurant Customers

Each customer is present during [a_i, b_i]. Find the maximum number present at once. Create an event (a_i, +1) and an event (b_i, −1) for each customer, sort the events by time, and keep a running sum. The maximum running sum is the answer. In CSES all times are distinct. When ties are possible, decide whether a departure and an arrival at the same moment overlap, and order the events accordingly. Pairs sort (t, −1) before (t, +1), which treats them as **not** overlapping.

5. Reference implementations

5.1 Movie Festival (CSES 1629)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<int, int>> mv(n);              // {end, start}
    for (auto &[e, s] : mv) cin >> s >> e;
    sort(mv.begin(), mv.end());
    int count = 0, lastEnd = 0;                // all times are >= 1
    for (auto [e, s] : mv) {
        if (s >= lastEnd) {
            count++;
            lastEnd = e;
        }
    }
    cout << count << '\n';
    return 0;
}
```

5.2 Ferris Wheel (CSES 1090)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    cin >> n >> x;
    vector<long long> p(n);
    for (auto &w : p) cin >> w;
    sort(p.begin(), p.end());
    int i = 0, j = n - 1, gondolas = 0;
    while (i <= j) {
        if (i < j && p[i] + p[j] <= x) i++;    // lightest joins heaviest
        j--;                                    // heaviest always boards
        gondolas++;
    }
    cout << gondolas << '\n';
    return 0;
}
```

5.3 Tasks and Deadlines (CSES 1630)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<long long, long long>> t(n);   // {duration, deadline}
    for (auto &[a, d] : t) cin >> a >> d;
    sort(t.begin(), t.end());                  // shortest job first
    long long time = 0, reward = 0;
    for (auto [a, d] : t) {
        time += a;
        reward += d - time;
    }
    cout << reward << '\n';
    return 0;
}
```

5.4 Largest concatenation

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<string> s(n);
    for (auto &x : s) cin >> x;
    sort(s.begin(), s.end(), [](const string &x, const string &y) {
        return x + y > y + x;
    });
    string res;
    for (auto &x : s) res += x;
    if (res[0] == '0') res = "0";              // all zeros
    cout << res << '\n';
    return 0;
}
```

On input `5` / `3 30 34 5 9` this prints `9534330`.

6. Common mistakes

1. **`<=` in a comparator.** Undefined behavior, which in practice means crashes on inputs with many equal keys.

2. **Greedy without proof.** Sorting by the "obvious" key (start time, size, ratio) and submitting. Always try to break it with a tiny case first.
3. **Ignoring ties.** Equal end times, simultaneous arrival and departure, equal weights. Decide explicitly what should happen.
4. **Overflow** in sums of finish times or costs. Use `long long`.
5. **Expensive comparators.** Comparing big structs by value copies them in every comparison. Take `const T&`. Building strings inside a comparator (as in the concatenation trick) is fine for small n but costs O(length) per comparison.
6. **Off-by-one in interval conditions.** Does a movie that starts exactly when another ends conflict with it? Read the statement.
7. **Sorting when only the k-th element is needed.** `nth_element` is O(n), though sorting is usually fast enough anyway.
8. **Losing original indices.** When output must refer to input positions, sort `{value, index}` pairs or sort indices.

7. Practice problems

CSES (Sorting and Searching section)

- 1629 — Movie Festival — https://cses.fi/problemset/task/1629
- 1090 — Ferris Wheel — https://cses.fi/problemset/task/1090
- 1074 — Stick Lengths — https://cses.fi/problemset/task/1074
- 1619 — Restaurant Customers — https://cses.fi/problemset/task/1619
- 1630 — Tasks and Deadlines — https://cses.fi/problemset/task/1630
- 1084 — Apartments — https://cses.fi/problemset/task/1084

AtCoder

- ABC131 D — Megalomania — https://atcoder.jp/contests/abc131/tasks/abc131_d
- ABC085 B — Kagami Mochi — https://atcoder.jp/contests/abc085/tasks/abc085_b

Codeforces

- 230A — Dragons — https://codeforces.com/problemset/problem/230/A
- 451B — Sort the Array — https://codeforces.com/problemset/problem/451/B
- 1353B — Two Arrays And Swaps — https://codeforces.com/problemset/problem/1353/B
- 545D — Queue — https://codeforces.com/problemset/problem/545/D
- 1029C — Maximal Intersection — https://codeforces.com/problemset/problem/1029/C

Kattis

- classrooms — https://open.kattis.com/problems/classrooms (interval scheduling with k rooms; combine this week's greedy with a `multiset`)

8. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 3 ("Sorting") and Chapter 6 ("Greedy algorithms"): https://cses.fi/book/book.pdf
- Jon Kleinberg and Éva Tardos, *Algorithm Design*, Chapter 4 ("Greedy Algorithms"), the clearest treatment of exchange arguments and "greedy stays ahead"
- cppreference, "Compare" named requirement (strict weak ordering): https://en.cppreference.com/w/cpp/named_req/Compare
- USACO Guide, Silver modules "Custom Comparators and Coordinate Compression" and "Greedy Algorithms with Sorting": https://usaco.guide/